

Markowitz Portfolio Optimisation Using a From-Scratch Implementation of Wolfe's Modified Simplex Method for Quadratic Programming

Debdan Samanta, Musaib Bin Bashir, Srijan Ghosh
Department of Industrial Engineering
Operations Research II

Abstract—This paper presents a complete, ground-up implementation of the Markowitz Mean-Variance Portfolio Optimisation model, solved entirely without the use of any external optimisation library. The core engine is built on Wolfe's Modified Simplex Method, a classical technique from 1959 that converts a convex Quadratic Program (QP) into an augmented Linear Program via the Karush–Kuhn–Tucker (KKT) first-order optimality conditions, and then solves the resulting system using a modified Simplex pivot procedure that enforces complementary slackness at every iteration. The implementation is applied to a real-world portfolio of nine Nifty50 equities and a Gold ETF, with live price data fetched from Yahoo Finance. Experimental results demonstrate that the solver correctly traces the Markowitz Efficient Frontier and produces allocation weights consistent with theoretical expectations.

Index Terms—Convex Quadratic Programming, Efficient Frontier, KKT Conditions, Markowitz Portfolio Theory, Wolfe's Modified Simplex Method.

I. INTRODUCTION

One of the most well-known problems in quantitative finance is the Markowitz portfolio selection problem [1]. The core idea is surprisingly intuitive: given a set of assets, each with some expected return and some risk (measured as variance), find the combination of assets that achieves the lowest possible risk for a desired level of return. This is not just a finance problem — it is, at its heart, a constrained optimisation problem, specifically a Quadratic Program.

Most practical implementations of portfolio optimisers rely on off-the-shelf solvers such as `scipy.optimize.minimize` or commercial tools like CVXPY. The goal of this project, however, was to implement the entire solver pipeline from scratch using only NumPy, in order to understand what is actually happening under the hood. This turned out to be significantly more involved than initially expected.

The solver used here is Wolfe's Modified Simplex Method [2], published in *Econometrica* in 1959. Wolfe's key insight was that the KKT conditions of a convex QP form a structured linear system, and that a specially modified version of the Simplex algorithm — one that respects the complementarity constraints at every pivot step — can solve it exactly. This paper documents the mathematical foundations, the algorithmic design, and the implementation decisions made throughout this project.

II. PROBLEM FORMULATION

A. The Markowitz Mean-Variance Model

Let there be n assets available for investment. The decision variable is the weight vector $w \in \mathbb{R}^n$, where each w_i represents the percentage of total capital allocated to asset i . The relevant parameters are:

- $\Sigma \in \mathbb{R}^{n \times n}$: the annualised covariance matrix of asset returns (in units of $\%^2$), which encodes both individual asset risk and pairwise correlations.
- $r \in \mathbb{R}^n$: the vector of annualised expected returns (in %).
- $T \in \mathbb{R}$: the investor's target minimum annualised return (in %).

The portfolio optimisation problem is stated as follows:

$$\begin{aligned} \min_w \quad & w^\top \Sigma w \\ \text{subject to} \quad & \mathbf{1}^\top w = 100 \\ & r^\top w \geq 100 \cdot T \\ & w_i \geq 0, \quad \forall i \in \{1, \dots, n\} \end{aligned} \tag{1}$$

The objective minimises portfolio variance, which is a direct measure of risk. The first constraint enforces full capital deployment (no cash position). The second constraint requires that the portfolio achieves at least the target return. The third constraint enforces that no asset is shorted (long-only positions).

Note that $\Sigma \succeq 0$ (positive semi-definite) by construction, which means the objective is convex. This is an important property because it guarantees that any locally optimal solution found is also globally optimal.

B. Reformulation as a Canonical QP

Wolfe's solver requires the problem to be in the following standard form:

$$\min_{x \geq 0} \quad \frac{1}{2} x^\top Q x + c^\top x \quad \text{s.t.} \quad A_{\text{eq}} x = b_{\text{eq}}, \quad A_{\text{ub}} x \leq b_{\text{ub}} \tag{2}$$

The portfolio problem maps to this canonical form by setting:

$$Q = 2\Sigma, \quad c = \mathbf{0}, \quad A_{\text{eq}} = \mathbf{1}^\top, \quad b_{\text{eq}} = 100 \tag{3}$$

The return constraint $r^\top w \geq 100T$ is converted to $\leq$ form by negating both sides:

$$A_{ub} = -r^\top, \quad b_{ub} = -100T \quad (4)$$

The factor of 2 in $Q = 2\Sigma$ is included because the solver minimises $\frac{1}{2}x^\top Qx$, so $\frac{1}{2}w^\top(2\Sigma)w = w^\top\Sigma w$, which is exactly what we want.

III. WOLFE'S MODIFIED SIMPLEX METHOD

A. Motivation

The standard Simplex algorithm is designed for Linear Programs (LPs) — problems where both the objective and constraints are linear. The Markowitz problem has a quadratic objective, so it cannot be solved directly with Simplex. Wolfe's contribution was to transform the QP into an equivalent LP-like system by leveraging the KKT optimality conditions.

B. KKT Conditions

For a convex QP, the KKT conditions are both necessary and sufficient for global optimality [3]. The Lagrangian of the canonical QP is:

$$\begin{aligned} \mathcal{L}(w, \lambda, \mu, s) = & \frac{1}{2}w^\top Qw + c^\top w \\ & + \lambda^\top (A_{eq}w - b_{eq}) \\ & + \mu^\top (A_{ub}w - b_{ub}) - s^\top w \end{aligned} \quad (5)$$

where $\lambda \in \mathbb{R}^p$ are the (free) Lagrange multipliers for the equality constraints, $\mu \in \mathbb{R}_{\geq 0}^q$ are the multipliers for the inequality constraints, and $s \in \mathbb{R}_{\geq 0}^n$ are the dual surplus variables associated with the non-negativity constraint $w \geq 0$.

Taking the gradient with respect to w and setting it to zero gives the *stationarity condition*:

$$Qw + c + A_{eq}^\top \lambda + A_{ub}^\top \mu - s = 0 \quad (6)$$

The *complementary slackness conditions* state:

$$s_i \cdot w_i = 0 \quad \forall i, \quad \mu_j \cdot t_j = 0 \quad \forall j \quad (7)$$

where $t_j = b_{ub,j} - (A_{ub}w)_j \geq 0$ is the slack variable for the j -th inequality. These conditions state that for each complementary pair, at most one of the two variables can be nonzero at optimality.

C. The Augmented Tableau

Wolfe's method constructs an augmented LP tableau whose rows encode the full KKT system. The variables in column order are listed in Table I.

The rows of the tableau encode the full KKT system:

$$Qw - s + A_{eq}^\top(\lambda^+ - \lambda^-) + A_{ub}^\top \mu = -c \quad (8)$$

$$A_{eq} w = b_{eq} \quad (9)$$

$$A_{ub} w + t = b_{ub} \quad (10)$$

This system, when solved with $w, s, \mu, t \geq 0$ and the complementarity conditions satisfied, yields the optimal solution to the original QP.

TABLE I
TABLEAU VARIABLE LAYOUT

Symbol	Count	Meaning
w	n	Primal decision variables
s	n	KKT surplus; complementary to w
λ^+, λ^-	p each	Positive/negative parts of the free multiplier $\lambda = \lambda^+ - \lambda^-$
μ	q	Inequality multipliers ≥ 0 ; complementary to t
t	q	Inequality slack variables; complementary to μ

D. Phase I: Big-M Initialisation

The Simplex method requires a starting basis — a set of m basic variables whose corresponding tableau columns form an identity matrix. For general QPs, no obvious initial basis exists. Wolfe's method uses the *Big-M technique* to handle this.

For every row that lacks a natural identity column, an *artificial variable* $a_i \geq 0$ is introduced. The objective is then augmented with a large penalty:

$$\min \quad \frac{1}{2}w^\top Qw + c^\top w + M \sum_i a_i, \quad M = 10^6 \quad (11)$$

This drives all artificials to zero during optimisation, effectively finding a feasible point for the KKT system. If any artificial remains in the basis at a nonzero value at termination, the original problem is infeasible.

In the portfolio problem specifically, since $c = \mathbf{0}$ and the return constraint has a negative RHS (which gets sign-flipped during tableau construction), all stationarity rows and all constraint rows require artificial variables.

E. The Complementarity Restriction

This is the key modification that distinguishes Wolfe's method from a vanilla Simplex. The standard pivot rule selects the entering variable as the one with the most negative reduced cost. Wolfe adds the following restriction:

Variable j may enter the basis only if its complementary partner is not currently in the basis.

The complementary pairs are (w_i, s_i) for all i , and (μ_j, t_j) for all j . Without this restriction, the Simplex method might produce a solution where both $w_i > 0$ and $s_i > 0$ simultaneously, which would violate the complementary slackness condition (7) and would not be a valid KKT point.

A two-pass selection procedure is used in practice:

- 1) **Pass 1:** Find the most negative reduced cost among all variables whose complementary partner is not in the basis.
- 2) **Pass 2:** If no such variable exists, check whether any blocked variable can enter via a degenerate pivot — specifically, one where the complementary partner would simultaneously leave the basis (its min-ratio equals zero). This handles degenerate cases without getting stuck.

F. Pivot Step

Once an entering variable is chosen, the leaving variable is determined by the standard *minimum ratio test*:

$$\text{leave} = \arg \min_{i : \text{col}[i] > 0} \frac{\text{RHS}[i]}{\text{col}[i]} \quad (12)$$

The pivot operation performs row reduction to restore the basis columns to standard form, exactly as in standard Simplex.

G. Termination Criteria

The algorithm terminates under one of three conditions:

- 1) **Optimal**: All reduced costs are non-negative and no eligible entering variable exists.
- 2) **Infeasible**: An artificial variable remains in the basis with a positive value, meaning the constraints are contradictory.
- 3) **Unbounded**: No positive entry exists in the entering column, meaning the objective can decrease without bound. This should not occur for a convex QP with a bounded feasible region.

IV. IMPLEMENTATION

A. Numerical Stability

Positive Semi-Definiteness. Historical covariance matrices estimated from finite data samples can have small negative eigenvalues due to sampling noise, which would make the QP non-convex. To handle this, a spectral decomposition is performed and any negative eigenvalues are clipped to zero:

$$\Sigma \leftarrow V \cdot \max(\Lambda, 0) \cdot V^\top \quad (13)$$

where $\Sigma = V\Lambda V^\top$ is the eigendecomposition. This yields the nearest PSD matrix to the original in the Frobenius norm sense.

Symmetry Enforcement. Floating-point arithmetic can introduce tiny asymmetries in Σ and Q . Both matrices are explicitly symmetrised at multiple points via $Q \leftarrow \frac{1}{2}(Q + Q^\top)$.

Weight Scaling. The solver works with weights summing to 100 (percent) rather than 1 (fraction). This improves the numerical conditioning of the tableau by keeping all matrix entries and RHS values in the range 10^1 – 10^2 , which reduces the relative effect of floating-point rounding during pivot operations.

Tolerance Constants. Three key tolerances govern numerical precision and are listed in Table II.

TABLE II
NUMERICAL TOLERANCE CONSTANTS

Constant	Value	Role
<code>_BIG_M</code>	10^6	Penalty weight for artificial variables
<code>_PIVOT_TOL</code>	10^{-10}	Minimum valid pivot element magnitude
<code>_FEAS_TOL</code>	10^{-8}	Zero threshold for artificial variables

B. Software Architecture

The implementation is structured as three independent modules with a clean dependency hierarchy:

- `QuadraticSimplex.py`: A general-purpose QP solver exposing a single public function `solve_qp_wolfe()`. It has no project-specific dependencies and can be reused for any convex QP.
- `PortfolioOptimizer.py`: Translates the Markowitz problem into the canonical QP form, enforces PSD on the covariance matrix, and wraps the solver output into structured result objects.
- `Task.py`: The top-level runner that fetches live market data, computes statistics, invokes the optimizer, and produces visualisations.

The dependency direction is strictly one-way: `Task.py` $\rightarrow$ `PortfolioOptimizer.py` $\rightarrow$ `QuadraticSimplex.py`.

C. Data Pipeline

Live daily adjusted-close prices are fetched from Yahoo Finance via the `yfinance` library for the most recent four months. *Log returns* are computed as $r_t = \ln(P_t/P_{t-1})$, preferred over simple returns because they are time-additive across periods and better approximate a normal distribution for short intervals — a key assumption of the Markowitz model. Annualised statistics are obtained by scaling by 252 trading days per year:

$$\hat{\mu}_{\text{ann}} = 252 \cdot \bar{r}_{\text{daily}}, \quad \hat{\Sigma}_{\text{ann}} = 252 \cdot \hat{\Sigma}_{\text{daily}} \quad (14)$$

Missing data (caused by thin trading or recent IPOs) is handled by backward-then-forward filling nearest known prices. Any asset with irrecoverable missing data is dropped from the universe entirely.

V. RESULTS

The solver was applied to a universe of nine Nifty50 equities (RELIANCE, TCS, HDFCBANK, INFY, ICICIBANK, HINDUNILVR, SBIN, BAJFINANCE, WIPRO) plus the Gold-BEES ETF, using four months of daily price history.

For a user-specified target return T , the solver produces the minimum-variance portfolio allocation. The efficient frontier is traced by solving the QP for 50 evenly-spaced target return values between the minimum and maximum feasible returns. The output is a three-panel visualisation:

- **Left panel**: The efficient frontier curve in (σ_p, μ_p) space, with individual assets plotted at their own risk-return coordinates and the optimal portfolio marked with a star.
- **Centre panel**: A bar chart of the optimal weight allocation for the chosen target T .
- **Right panel**: The evolution of each asset's weight along the frontier, showing how diversification shifts as the target return increases.

The solver correctly finds corner solutions (100% allocation to a single asset) at extreme target returns, and interior

diversified allocations at intermediate returns, consistent with theoretical expectations from [1].

To illustrate the solver’s practical output, two specific target annualised returns were evaluated: $T = 10\%$ and $T = 15\%$. The resulting optimal portfolio allocations and their associated risk statistics are summarised in Table III. Both scenarios yield diversified portfolios across RELIANCE, SBIN, and Gold, with higher risk targets driving more aggressive allocations.

TABLE III
OPTIMAL PORTFOLIOS FOR $T = 10\%$ AND $T = 15\%$ TARGETS

Metric / Asset Weight	$T = 10\%$	$T = 15\%$
Portfolio Return	10.00%	15.00%
Portfolio Std Dev (σ)	20.88%	22.65%
Portfolio Variance (σ^2)	436.17 % ²	512.85 % ²
Sharpe Ratio ($r_f = 0$)	0.4788	0.6624
RELIANCE	19.40%	8.94%
SBIN	65.40%	73.90%
Gold	15.21%	17.16%
All other assets	0.00%	0.00%

VI. CONCLUSION

This project demonstrates that the Markowitz portfolio optimisation problem can be solved from first principles, without any external QP library, by implementing Wolfe’s Modified Simplex Method directly. The key contributions of this implementation are: (i) a complete Big-M Phase I initialisation that dynamically identifies which rows require artificial variables; (ii) a two-pass complementarity restriction that correctly handles degenerate pivot cases; and (iii) a numerically stable problem formulation with PSD enforcement and percentage-unit scaling.

One limitation worth noting is that the Big-M method, while straightforward, can sometimes suffer from numerical issues when M is not large enough to force artificials out, or when it interacts poorly with the problem’s natural scaling. A more robust alternative would be a two-phase Simplex that separates Phase I (feasibility) from Phase II (optimisation) entirely. That said, for the problem sizes encountered in this project (10 assets, 2 constraints), the Big-M approach performs reliably.

REFERENCES

- [1] H. Markowitz, “Portfolio Selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, Mar. 1952.
- [2] P. Wolfe, “The Simplex Method for Quadratic Programming,” *Econometrica*, vol. 27, no. 3, pp. 382–398, Jul. 1959.
- [3] M. S. Bazaraa, H. D. Sherali, and C. M. Shetty, *Nonlinear Programming: Theory and Algorithms*, 3rd ed. Hoboken, NJ, USA: Wiley, 2006, ch. 4.
- [4] E. K. P. Chong and S. H. Zak, *An Introduction to Optimization*, 4th ed. Hoboken, NJ, USA: Wiley, 2013, ch. 17.
- [5] R. Aroussi, “yfinance: Download market data from Yahoo! Finance,” Python Package Index (PyPI). [Online]. Available: <https://pypi.org/project/yfinance/>